

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание классов

Студент гр. 3343

Старков С.А

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2024

Цель работы

Разработать систему классов для моделирования игры "Морской бой", включая классы кораблей, менеджера кораблей и игрового поля. Классы должны обеспечивать корректное размещение кораблей на поле, обработку атак и отслеживание состояния кораблей и поля.

Задание

а. Создать класс корабля, который будет размещаться на игровом поле. Корабль может иметь длину от 1 до 4, а также может быть расположен вертикально или горизонтально. Каждый сегмент корабля может иметь три различных состояния: целый, поврежден, уничтожен. Изначально у корабля все сегменты целые. При нанесении 1 урона по сегменту, он становится поврежденным, а при нанесении 2 урона по сегменту, уничтоженным. Также добавить методы для взаимодействия с кораблем.

б. Создать класс менеджера кораблей, хранящий информацию о кораблях. Данный класс в конструкторе принимает количество кораблей и их размеры, которые нужно расставить на поле.

с. Создать класс игрового поля, которое в конструкторе принимает размеры. У поля должен быть метод, принимающий корабль, координаты, на которые нужно поставить, и его ориентацию на поле. Корабли на поле не могут соприкасаться или пересекаться. Для игрового поля добавить методы для указания того, какая клетка атакуется. При попадании в сегмент корабля изменения должны отображаться в менеджере кораблей.

Каждая клетка игрового поля имеет три статуса:

- i. неизвестно (изначально вражеское поле полностью неизвестно),
- ii. пустая (если на клетке ничего нет)
- iii. корабль (если в клетке находится один из сегментов корабля).

Для класса игрового поля также необходимо реализовать конструкторы копирования и перемещения, а также соответствующие им операторы присваивания.

Примечания:

- Не забывайте для полей и методов определять модификаторы доступа
- Для обозначения переменной, которая принимает небольшое ограниченное количество значений, используйте `enum`
- Не используйте глобальные переменные

- При реализации копирования нужно выполнять глубокое копирование
- При реализации перемещения, не должно быть лишнего копирования
- При выделении памяти делайте проверку на переданные значения
- У поля не должно быть методов возвращающих указатель на поле в явном виде, так как это небезопасно

Выполнение работы

В работе представлено несколько классов: **Game**, **Map**, **Ship**, **ShipManager** и структуры **Cell**, **Segment**, **Coordinate**.

ShipManager отвечает за управление массивом кораблей. Метод **createShip()** создает стандартное количество кораблей для игры в морской бой на поле размером 10 на 10. При размещении корабля на игровом поле, поле **isPlaced** меняет значение на **true**. Игровое поле представлено массивом **FieldCell**, каждая из которых хранит информацию о том, есть ли на ней сегмент корабля, свои координаты и статус (раскрыта ли она).

На клетку можно установить корабль, если рядом нет других кораблей. Если корабль успешно установлен, **FieldCell** содержит указатель на сегмент корабля. При атаке клетки она становится раскрытой, а статус сегмента меняется (изначально сегмент целый; чтобы его уничтожить, необходимо нанести две атаки). Изменение статуса отображается в **ShipManager**, поскольку в клетке хранится указатель на сегмент корабля. Если атака производится на пустую клетку, она раскрывается, а её статус меняется на промах.

Корабли можно расставить случайным образом, передавая в метод **setAllShips()** массив кораблей из **ShipManager**. Эта функция генерирует случайные координаты и ориентацию для корабля, передает это вместе с указателем на корабль в метод **setShip()**, пока не будут установлены все корабли.

Поля класса **Map**:

1. **width**: Ширина игрового поля.
2. **height**: Высота игрового поля.
3. **field**: Массив из **FieldCell**, представляющий игровое поле. Каждая клетка содержит координаты, значение (**empty**, **miss**, **shipPart**, **hit**, **destroyed**) и статус (**HIDDEN**, **DISCLOSED**).

Методы класса **Map**:

1. **Map(int width, int height):** Конструктор, который инициализирует игровое поле заданной ширины и высоты, создавая вектор **field** из объектов **FieldCell**.
2. **placeShip(Ship&, Coordinate, Orientation):** Метод, который устанавливает корабль на игровом поле по заданным координатам и ориентации.
3. **display() const:** Метод для отображения игрового поля.
4. **attackCell(Coordinate):** Метод, который атакует клетку игрового поля по заданным координатам.

Класс **Ship** представляет отдельный корабль. Он отвечает за хранение сегментов и координат корабля.

Поля класса Ship:

1. **segments:** Вектор указателей на сегменты корабля (объекты типа **Segment**).
2. **length:** Длина корабля (целое число от 1 до 4).
3. **isPlaced:** Флаг, указывающий, размещен ли корабль на игровом поле (по умолчанию false).
4. **isVertical:** Флаг, указывающий, расположен ли корабль вертикально (по умолчанию false).

Методы класса Ship:

1. **Ship(int shipLength):** Конструктор, который инициализирует корабль заданной длины. Если длина корабля не находится в диапазоне от 1 до 4, выбрасывается исключение.
2. **~Ship():** Деструктор, освобождающий память, выделенную для сегментов.
3. **getLength(), getIsVertical(), getSegments(), getIsPlaced():** Методы доступа, возвращающие соответствующие значения.

Класс **ShipManager** управляет вектором кораблей на игровом поле и отвечает за их создание и хранение.

Поля класса ShipManager:

1. **ships:** Вектор указателей на объекты типа **Ship**.

Методы класса ShipManager:

1. **ShipManager()**: Конструктор, создающий пустой вектор указателей на корабли.
2. **createDefaultShips()**: Метод, который создает вектор кораблей с заданными длинами {4, 3, 3, 2, 2, 2, 1, 1, 1, 1}.

Структуры:

- **Segment**: Структура, представляющая сегмент корабля с координатами и статусом.
- **Cell**: Структура, представляющая ячейку игрового поля с координатами, статусом и значением.
- **Coordinate**: Структура, представляющая координаты x и y с оператором сравнения.

Разработанный программный код см. в приложении А.

Тестирование

1. Создается объект **Map** размером 10x10.
2. Создается объект **ShipManager** с заданными размерами кораблей {3, 2, 1}.
3. Вызывается метод **placeShip()** объекта **Map** для размещения корабля на координатах (5, 3) в горизонтальном положении.
4. Вызывается метод **display()** объекта **Map**, который отображает текущее состояние игрового поля.
5. Вызывается метод **attackCell()** объекта **Map** для атаки клетки с координатами (5, 3).
6. Снова вызывается метод **display()** объекта **Map**, который обновляет и отображает состояние игрового поля после атаки.
7. Выполняется атака клетки с координатами (9, 9) методом **attackCell()** объекта **Map**.
8. Создается новый объект **Ship** длиной 3.
9. Вызывается метод **placeShip()** объекта **Map** для размещения нового корабля на координатах (0, 0) в вертикальном положении.
10. Вызывается метод **display()** объекта **Map** для отображения игрового поля с новыми изменениями.

Тестирование проверяет корректность размещения кораблей, атаку клеток и отображение состояния игрового поля.

```

void Game::gameInit(){
    std::cout << "Game initialized!" << std::endl;
    Map map( width: Constants::BOARD_SIZE, height: Constants::BOARD_SIZE);
    std::vector<int> sizes = {3,2,1};
    ShipManager shipManager( shipSizes: sizes);

    Ship ship( length: 3);
    map.placeShip( & ship, startPos: Coordinate( x_val: 5 , y_val: 3), orientation: Orientation::HORIZONTAL);
    map.display();
    // map.printCell(Coordinate(5 ,3));
    map.attackCell( coord: Coordinate( x_val: 5, y_val: 3));
    map.display();
    // map.printCell(Coordinate(5 ,3));
    map.attackCell( coord: Coordinate( x_val: 9, y_val: 9));

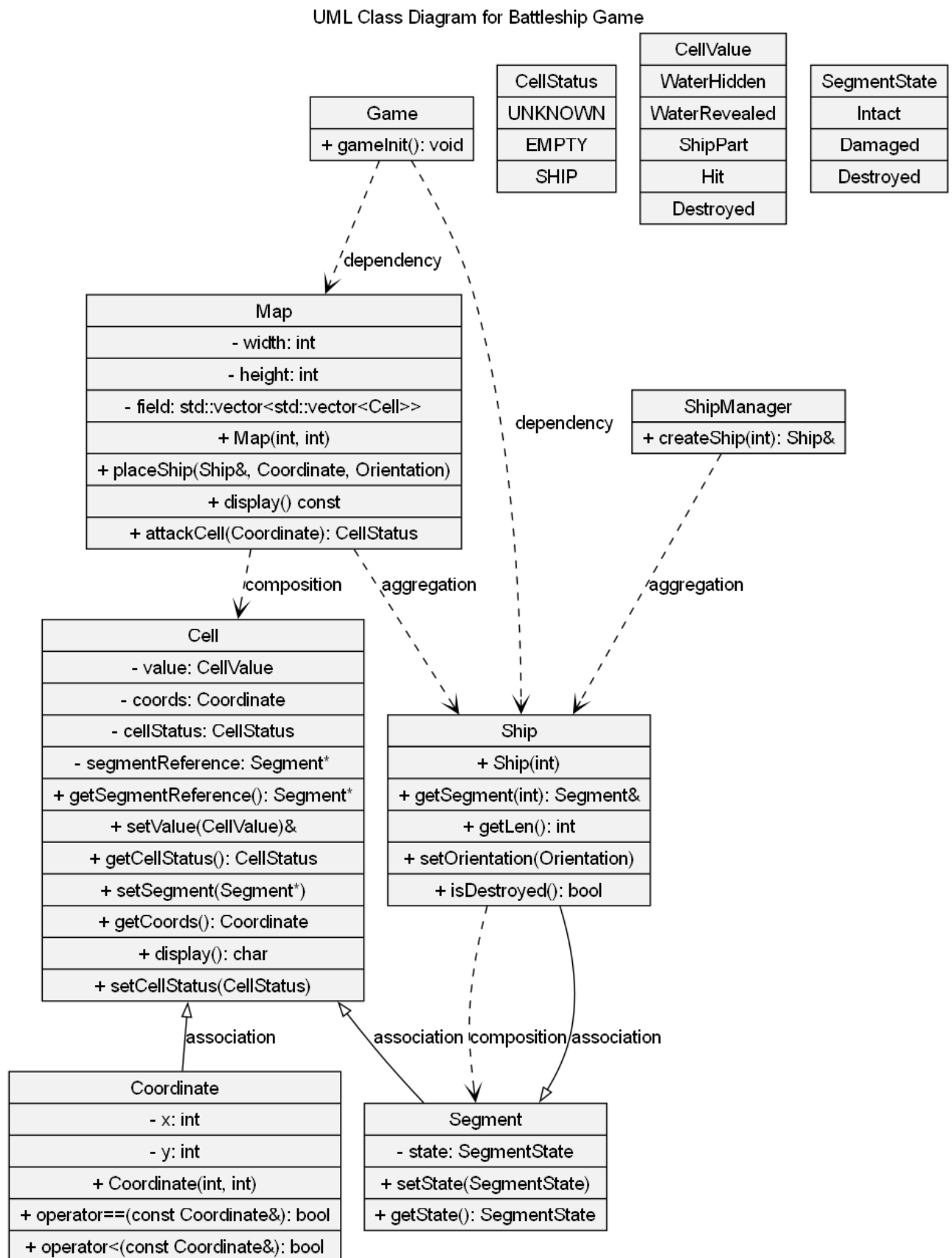
    Ship ship1( length: 3);
    map.placeShip( & ship1, startPos: Coordinate( x_val: 0, y_val: 0), orientation: Orientation::VERTICAL);
    map.display();
    // map.placeShip(Co
};

```

Вывод программы:

		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
1.		S		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
2.		S		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
3.		S		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
4.		~		~		~		~		~		0		S		S		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
5.		~		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
6.		~		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
7.		~		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
8.		~		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
9.		~		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
10.		~		~		~		~		~		~		~		~		~		
		+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+	-	+
		1		2		3		4		5		6		7		8		9		A

Диаграмма классов:



Выводы

В ходе выполнения лабораторной работы, был реализован функционал добавления кораблей, заданных размеров на игровое поле, создания игрового поля с заданными размерами, а также атаку по клетке поля и повреждение сегмента. Корабля, при попадании в него. Были реализованы методы для демонстрации корректной работы программы и UML-диаграмма классов.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include "GameField.hpp"
#include "ShipManager.hpp"

int main()
{
    GameField gf(10, 10);
    ShipManager sh;
    gf.setAllShips(sh.getShips());
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            gf.attackCell(Coordinates{ i,j });
            sh.registerDamage(Coordinates{ i,j });
        }
    }
    sh.printShipsInfo();
    gf.drawField();
}
```

Название файла: GameField.cpp

```
##include "GameField.hpp"
#include <random>
#include <chrono>

GameField::GameField(int gf_width, int gf_height) {
    width = gf_width;
    height = gf_height;
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            field.push_back(FieldCell{ Coordinates{x,y},
CellStatus::HIDDEN, CellValue::Empty });
        }
    }
}
```

```

    }
}

GameField::GameField(const GameField& other)
    : height(other.height), width(other.width), field(other.field)
{}

GameField& GameField::operator=(const GameField& other) {
    if (this != &other) {
        width = other.width;
        height = other.height;
        field = other.field;
    }
    return *this;
}

GameField::GameField(GameField&& other)
    :          height(other.height),          width(other.width),
field(std::move(other.field)) {
    other.width = 0;
    other.height = 0;
    other.field.clear();
}

GameField& GameField::operator=(GameField&& other) {
    if (this != &other) {
        width = other.width;
        height = other.height;
        field = std::move(other.field);
        other.width = 0;
        other.height = 0;
    }
    return *this;
}

int  GameField::getWidth() {
    return width;
}

int  GameField::getHeight() {

```

```

        return height;
    }

    std::vector<FieldCell> GameField::getField() {
        return field;
    }

    bool GameField::checkCurrentCoord(int x, int y) {
        if (x<0 || x>width - 1 || y<0 || y>height - 1) {
            return false;
        }
        return true;
    }

    bool GameField::checkCoordsAround(int x, int y) {
        if (checkCurrentCoord(x, y)) {
            for (int i = -1; i <= 1; i++) {
                for (int j = -1; j <= 1; j++) {
                    if (checkCurrentCoord(x + i, y + j)) {
                        if (field[x + i + (y + j) *
width].shipSegment != nullptr) {
                            return false;
                        }
                    }
                }
            }
        }
        else return false;

        return true;
    }

    void GameField::setShip(Coordinates coords, Ship* ship, bool
isVertical) {
        if (!ship)
            return;
        bool ableToPlaceShip = true;
        if (checkCoordsAround(coords.x, coords.y)) {
            for (int i = 1; i < ship->getLength(); i++)
                {

```

```

        if (isVertical) {
            ableToPlaceShip = checkCoordsAround(coords.x,
coords.y + i);
        }
        else {
            ableToPlaceShip = checkCoordsAround(coords.x +
i, coords.y);
        }
        if (!ableToPlaceShip)
            return;
    }
    ship->getSegments()[0]->coord =
Coordinates{ coords.x ,coords.y };
    field[coords.x + coords.y * width].shipSegment =
ship->getSegments()[0];
    field[coords.x + (coords.y) * width].value =
CellValue::ShipSegment;
}
else {
    return;
}

ship->setIsVertical(isVertical);
ship->setIsPlaced(true);

if (isVertical) {
    //start point is up
    for (int i = 1; i < ship->getLength(); i++)
    {
        ship->getSegments()[i]->coord =
Coordinates{ coords.x ,coords.y + i };
        field[coords.x + (coords.y + i) * width].shipSegment
= ship->getSegments()[i];
        field[coords.x + (coords.y + i) * width].value =
CellValue::ShipSegment;
    }
}
else {
    //start point is left

```

```

        for (int i = 1; i < ship->getLength(); i++)
        {
            ship->getSegments()[i]->coord =
Coordinates{ coords.x + i, coords.y };
            field[coords.x + i + (coords.y * width)].shipSegment
= ship->getSegments()[i];
            field[coords.x + i + (coords.y * width)].value =
CellValue::ShipSegment;
        }
    }
}

void GameField::setAllShips(std::vector<Ship*> ships) {
    for (auto& ship : ships) {
        while (!ship->getIsPlaced()) {

            std::mt19937
gen(std::chrono::steady_clock::now().time_since_epoch().count());
            std::uniform_int_distribution<int> distr(0, 9);
            std::uniform_int_distribution<int> distr_bool(0, 1);

            int x = distr(gen);
            int y = distr(gen);
            bool random_bool =
static_cast<bool>(distr_bool(gen));

            this->setShip(Coordinates{ x, y }, ship,
random_bool);

        }
    }
}

void GameField::attackCell(Coordinates coords) {
    if (!checkCurrentCoord(coords.x, coords.y)) {
        return;
    }
    FieldCell& cell = field[coords.x + coords.y * width];
    cell.status = CellStatus::DISCLOSED;
}

```

```

switch (cell.value)
{
case CellValue::Empty:
    cell.value = CellValue::Miss;
    break;
case CellValue::ShipSegment: {
    if (cell.shipSegment->status == SegmentStatus::INTACT) {
        cell.shipSegment->status = SegmentStatus::DAMAGED;
    }
    else
if (cell.shipSegment->status==SegmentStatus::DAMAGED) {
        cell.shipSegment->status = SegmentStatus::DESTROYED;
    }
    break;
}
default:
    break;
}

```

}} **Название файла: GameField.hpp**

```

#pragma once
#include "Ship.hpp"
#include "Structures.hpp"
#include <vector>
#include <iostream>

class GameField {

private:
    int width;
    int height;
    std::vector<FieldCell> field;

public:
    GameField(int width,int height);
    GameField(const GameField& other);
    GameField& operator=(const GameField& other);

```

```

    GameField(GameField&& other);
    GameField& operator=(GameField&& other);
    bool checkCurrentCoord(int x,int y);
    bool checkCoordsAround(int x, int y);
    bool setShip(Coordinates coords, Ship* ship, bool isVertical);
    void setAllShips(std::vector<Ship*> ships);
    void attackCell(Coordinates coords);
};

```

Название файла: Structures.hpp

```
#pragma once
```

```
#include <iostream>
```

```
enum class CellStatus { HIDDEN, DISCLOSED };
```

```
enum class CellValue { Empty, Miss, ShipPart, Hit, Destroyed };
```

```
enum class SegmentStatus { INTACT, DAMAGED, DESTROYED };
```

```
struct Coordinates {
```

```
    int x;
```

```
    int y;
```

```
    bool operator==(const Coordinates& other) const {
        return x == other.x && y == other.y;
    }
};
```

```
struct FieldCell {
```

```
    Coordinates coord;
```

```
    CellStatus status;
```

```
    CellValue value;
```

```
};
```

```
struct ShipSegment {
```

```
    Coordinates coord;
```

```
    SegmentStatus status;
```

```
    ShipSegment() : coord({ 0, 0 }), status(SegmentStatus::INTACT) {}
```

```
    ShipSegment(Coordinates coord, SegmentStatus status) :
```

```
coord(coord), status(status) {}
```

```
}; Название файла:Ship.cpp
```

```

#include "Ship.hpp"
#include <iomanip>

Ship::Ship(int shipLength) : length(shipLength) {
    if (shipLength > 4 || shipLength < 1) {
        throw std::invalid_argument("Size must be in range [1,4]");
    }
};

Ship::~~Ship() {
    for (auto& segment : segments) {
        delete segment;
    }
}

int Ship::getLength() {
    return length;
}

bool Ship::getIsVertical() {
    return isVertical;
}

std::vector<ShipSegment*> Ship::getSegments() {
    return segments;
}

bool Ship::getIsPlaced() {
    return isPlaced;
}

Coordinates Ship::getCoords() {
    return coords;
}

void Ship::setIsPlaced(bool isPlaced) {

```

```

        this->isPlaced = isPlaced;
    }

    void Ship::setIsVertical(bool isVertical) {
        this->isVertical = isVertical;
    }

    void Ship::setCoords(Coordinates coords) {
        this->coords = coords;
    }

    void Ship::addSegment(ShipSegment* segment){
        segments.push_back(segment);
    }

```

Название файла: Ship.hpp

```

#pragma once
#include "Structures.hpp"
#include <string>
#include <vector>

class Ship {

private:
    int length;
    bool isPlaced = false;
    bool isVertical = false;
    Coordinates coords{0,0};
    std::vector<ShipSegment*> segments;

public:
    Ship(int shipLength);
    ~Ship();
    int getLength();
    std::vector<ShipSegment*> getSegments();
    bool getIsPlaced();
    bool getIsVertical();
    Coordinates getCoords();
    bool isVerticalOrientation();

```

```

        void setCoords(Coordinates coords);
        void setIsPlaced(bool isPlaced);
        void setIsVertical(bool isVertical);
        void addSegment(ShipSegment* segment);
};

```

Название файла: ShipManager.cpp

```

#include "ShipManager.hpp"
#include <iostream>

ShipManager::ShipManager() {
    std::vector<int> sizes = { 4, 3, 3, 2, 2, 2, 1, 1, 1, 1 };

    for (int i = 0; i < size(sizes); i++) {
        ships.push_back(new Ship (sizes[i]));
    }
}

ShipManager::~ShipManager() {
    for (auto& ship : ships) {
        delete ship;
    }
}

std::vector<Ship*> ShipManager::getShips() {
    return ships;
}

Ship& ShipManager::getShipByCoordinates(Coordinates coords) {
    for (auto& ship : ships) {
        if (ship->getCoords()==coords)
            return *ship;
    }
}

void ShipManager::printShipsInfo() {
    std::cout << "ships amount: " << size(ships)-1 << "\n\n";
}

```

```

        for (auto& ship:ships) {
            std::cout << "length: " << ship->getLength() << "\n";
            for (int i = 0; i < size(ship->getSegments()); i++)
            {
                std::cout << "Segment " << i << " coords x: " <<
ship->getSegments()[i]->coord.x
                << " y: " << ship->getSegments()[i]->coord.y<<"
status:";

                switch (ship->getSegments()[i]->status)
                {
                    case SegmentStatus::INTACT:
                        std::cout << "intact";
                        break;
                    case SegmentStatus::DAMAGED:
                        std::cout << "damaged";
                        break;
                    case SegmentStatus::DESTROYED:
                        std::cout << "destroyed";
                        break;
                    default:
                        break;
                }
                std::cout<< "\n";
            }
            std::cout << "\n";
        }
    }
}

```

```

void ShipManager::registerDamage(Coordinates hitCoords) {
    for (auto& ship : ships) {
        for (auto& segment : ship->getSegments()) {
            if (segment->coord == hitCoords) {
                if (segment->status == SegmentStatus::INTACT) {
                    segment->status = SegmentStatus::DAMAGED;
                }
                else if (segment->status == SegmentStatus::DAMAGED)
{
                    segment->status = SegmentStatus::DESTROYED;
                    bool isDestroyed = true;

```

```

        for (auto& currSegm : ship->getSegments()) {
            if (currSegm->status !=
SegmentStatus::DESTROYED) {
                isDestroyed = false;
            }
        }
        if (isDestroyed) {
            std::cout <<"ship on coords x: " <<
ship->getCoords().x
                << " y: " << ship->getCoords().y << " is
destroyed\n";
        }
    }
    else if (segment->status == SegmentStatus::DESTROYED)
{
        std::cout << "segment x: " << segment->coord.x
<< " y: " << segment->coord.y << " already destroyed\n";
    }
    return;
}
}
}

```

} Название файла: ShipManager.hpp

```
#pragma once
```

```
#include "Ship.hpp"
```

```
class ShipManager {
```

```
private:
```

```
    std::vector<Ship*> ships;
```

```
public:
```

```
    ShipManager();
```

```
    ~ShipManager();
```

```
    std::vector<Ship*> getShips();
```

```
    Ship& getShipByCoordinates(Coordinates coords);
```

```

    void printShipsInfo();
    void registerDamage(Coordinates hitCoords);
};

```

Название файла: ConsoleDisplayer.hpp

```

#include "GameField.hpp"
#include "ShipManager.hpp"
class ConsoleDisplayer {
public:
    void displayField(GameField& gf, CellStatus status);
    void displayShipsInfo(ShipManager& sm);
    inline void setColor(int foreground, int background = 40, int
attributes = 0);
    inline void resetColor();
};

```

Название файла: ConsoleDisplayer.cpp

```

#include "ConsoleDisplayer.hpp"

inline void ConsoleDisplayer::setColor(int foreground, int background , int
attributes ) {
    std::cout << "\033[" << attributes << ";" << foreground << ";" <<
background << "m";
}
inline void ConsoleDisplayer::resetColor() {
    std::cout << "\033[0m";
}

void ConsoleDisplayer::displayField(GameField& gf, CellStatus status) {
    std::cout << " ";
    for (int i = 0; i < gf.getWidth(); i++)
    {
        setColor(36);
        std::cout << "  " << i << " ";
    }
    resetColor();
    std::cout << "\n ";
    for (int i = 0; i < gf.getWidth(); i++)
    {
        std::cout << "+ - ";
    }
}

```

```

std::cout << "+ \n | ";
for (auto& cell : gf.getField()) {

    if (status == CellStatus::HIDDEN && cell.status ==
CellStatus::HIDDEN) {

        setColor(34);//BLUE
        std::cout << "~";
    }
    else {
        if (cell.status == CellStatus::DISCLOSED || status==
CellStatus::DISCLOSED)

            if (cell.value == CellValue::ShipSegment) {
                switch (cell.shipSegment->status) {
                    case SegmentStatus::INTACT:
                        setColor(32);//GREEN
                        std::cout << "O";
                        break;

                    case SegmentStatus::DAMAGED:
                        setColor(33);//YELLOW
                        std::cout << "O";
                        break;

                    case SegmentStatus::DESTROYED:
                        setColor(31);//RED
                        std::cout << "O";
                        break;
                    default:
                        break;
                }
            }
        else {
            switch (cell.value) {
                case CellValue::Empty:
                    setColor(34);//BLUE
                    std::cout << "#";
                    break;
                case CellValue::Miss:
                    setColor(31);//RED
                    std::cout << "x";
                    break;
                default:
                    break;
            }
        }
    }
}

```

```

        }
    }
}
resetColor();
if (cell.coord.x == gf.getWidth() - 1) {
    std::cout << " | ";
    setColor(36);
    std::cout << cell.coord.y;
    resetColor();
}
else
    std::cout << " | ";
if (cell.coord.x == gf.getWidth() - 1) {
    std::cout << "\n ";
    for (int i = 0; i < gf.getWidth(); i++)
    {
        std::cout << "+ - ";
    }
    std::cout << "+ ";
    if ((cell.coord.y) != gf.getHeight() - 1)
        std::cout << "\n | ";
}
}
std::cout << '\n';
}

```

```

void ConsoleDisplayer::displayShipsInfo(ShipManager& sm) {
    std::cout << "\nships amount: " << size(sm.getShips()) - 1 << "\n\n";
    for (auto& ship : sm.getShips()) {
        std::cout << "length: " << ship->getLength() << "\n";
        for (int i = 0; i < size(ship->getSegments()); i++)
        {
            std::cout << "Segment " << i << " coords x: " << ship-
>getSegments()[i]->coord.x
            << " y: " << ship->getSegments()[i]->coord.y << "
status:";

            switch (ship->getSegments()[i]->status)
            {
            case SegmentStatus::INTACT:
                std::cout << "intact";
                break;
            case SegmentStatus::DAMAGED:
                std::cout << "damaged";

```

```
        break;
    case SegmentStatus::DESTROYED:
        std::cout << "destroyed";
        break;
    default:
        break;
    }
    std::cout << "\n";
}
std::cout << "\n";
}
```